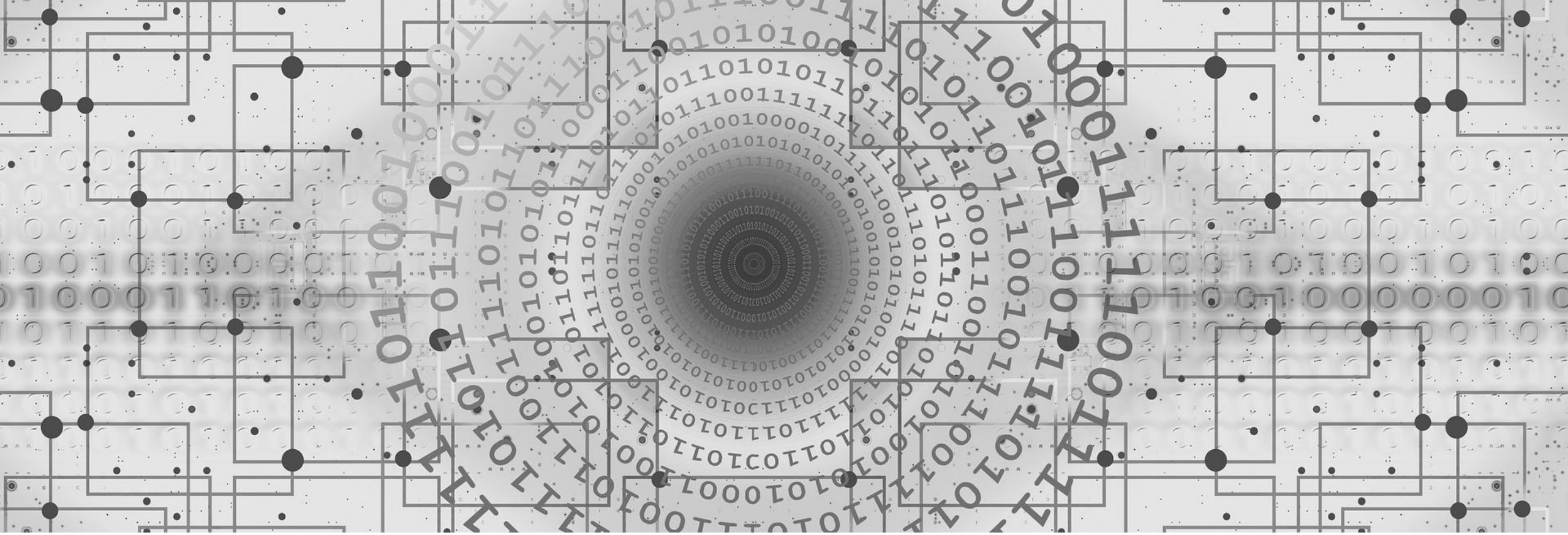


Homework-8 : Handshake Synchronizer

ECE-111 Advanced Digital Design project

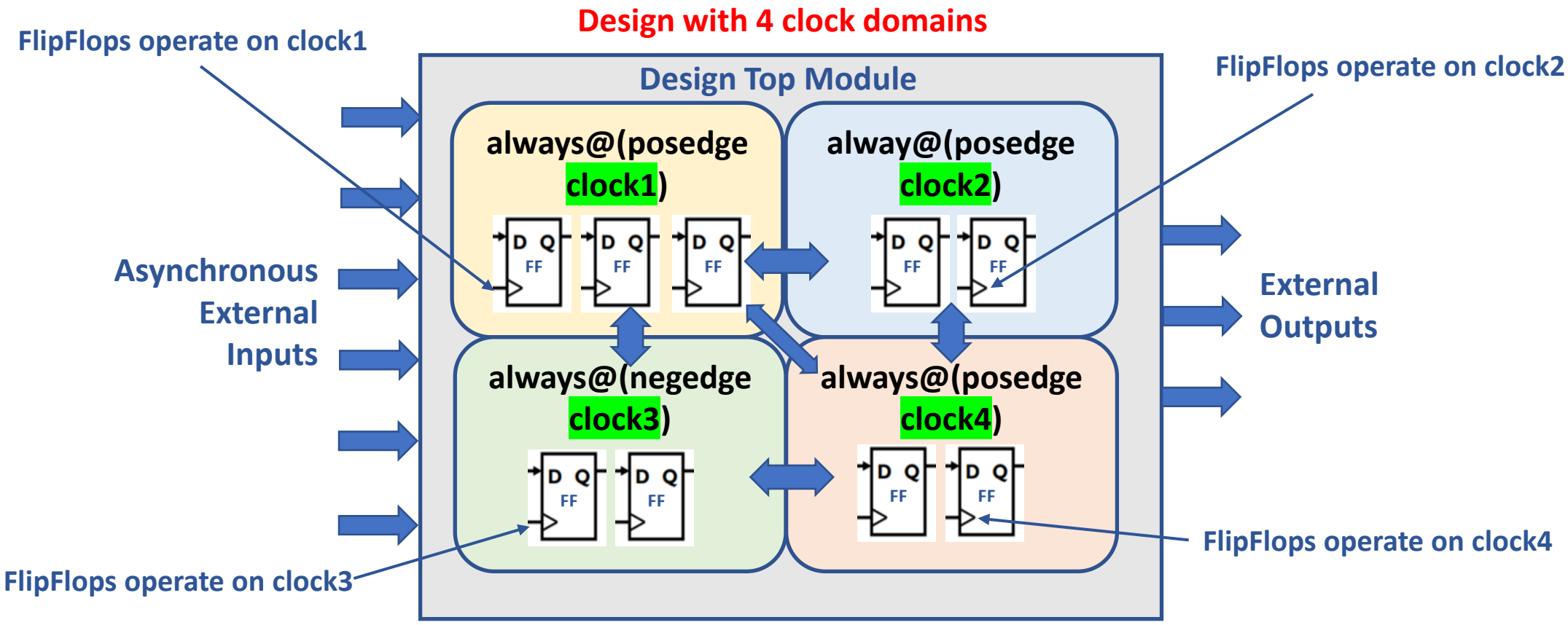
UC San Diego
JACOBS SCHOOL OF ENGINEERING
Electrical and Computer Engineering

Outline

- **First let's learn clock domain crossing (CDC) and data synchronization Concepts**
  - What is clock domain ?
  - What is clock domain crossing (CDC) ?
  - Synchronization issues when data flows from one clock domain to another clock domain
  - Data synchronization techniques
    - 2-FlipFlop Synchronizer
    - Handshake Synchronizer
  - Which synchronization technique to apply when data is sent from :
    - Slow to Fast clock domain
    - Fast to Slow clock domain

- **Homework-8 : Handshake Synchronizer**
  - Handshake synchronizer requirements including Sender and Receiver FSM Requirements
  - 4-phase handshake protocol
  - Block diagram of handshake synchronizer to understand overall data synchronization implementation
  - Simulation waveforms for reference purpose

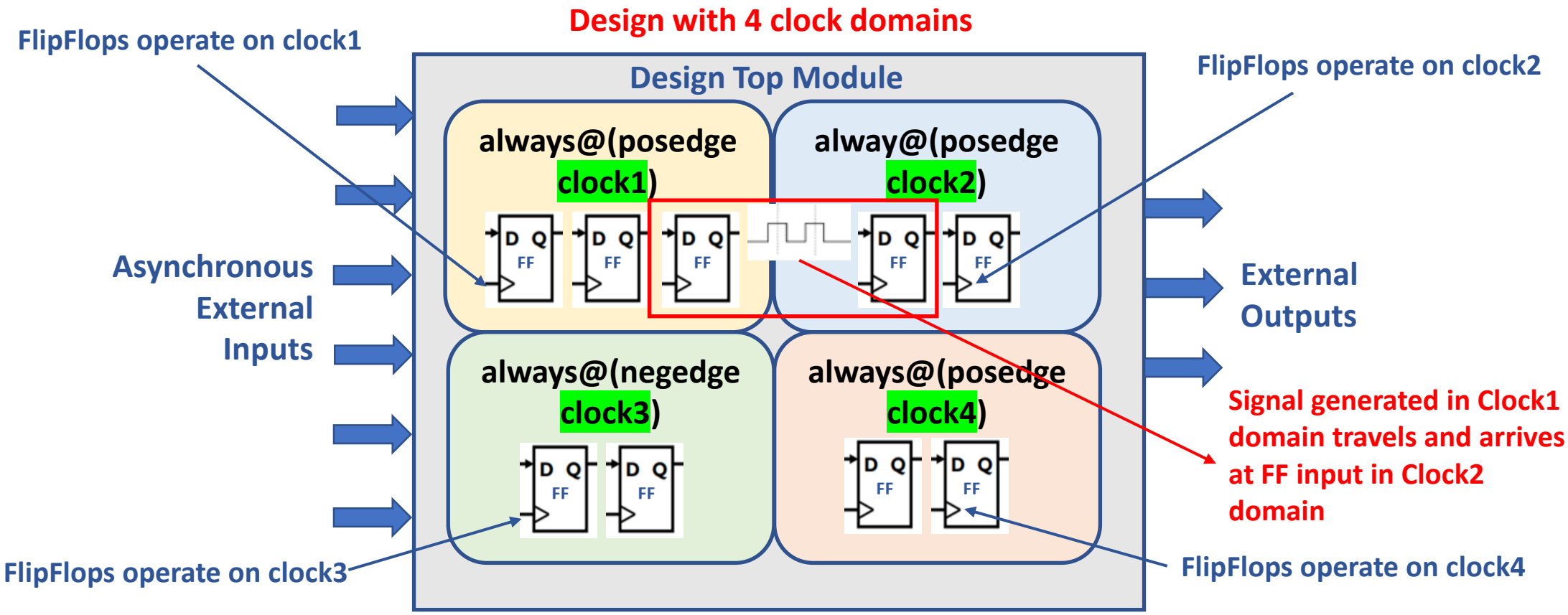
Clock Domain



❑ What is a Clock Domain ?

- **Clock Domain** is that portion of a circuit that is generated and processed by a single clock
- In diagram below, there are 4 clock domains in design top module. These clock domains are :
  - clock1, clock2, clock3 and clock4
  - All 4 clocks can be of different frequency, out of phase and asynchronous to each other
  - Flipflops in each clock domain operate on their respective clock

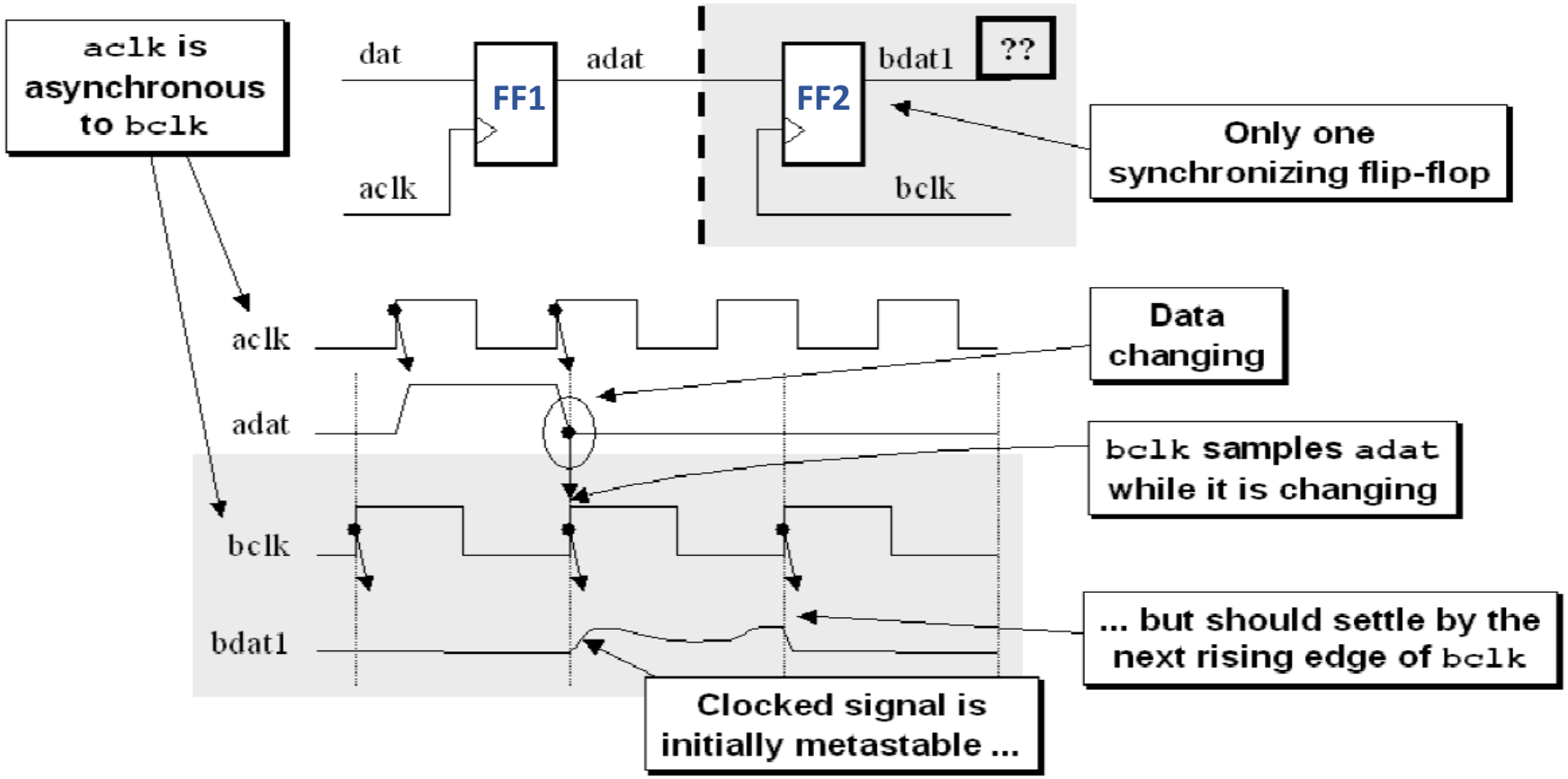
Clock Domain Crossing (CDC)



❑ What is Clock Domain Crossing (CDC) ?

- **Signal travels from one clock domain to another clock domain**
- **CDC** takes place anytime the inputs to a given flipflop were set based upon some other clock
- In example above signal generated from FF operating on clock1 travels and arrives at the flipflop inputs which is operating on clock2 → **Signal crossed clock domains**

What is the issue when signal crosses clock domains ?



- Synchronization failure that occurs when a signal (**adat**) generated in one clock domain (**aclk**) is sampled too close to the rising edge of a clock signal (**bclk**) from a second clock domain.
- **adat does not meet setup time requirement for FF2 hence output of FF2 (bdat1) goes metastable**
  - output (**bdat1**) going metastable and not converging to a legal stable state by the time the output must be sampled again.

Source of this slide courtesy of Clock Domain Crossing publication from Clifford E. Cummings Sunburst Design

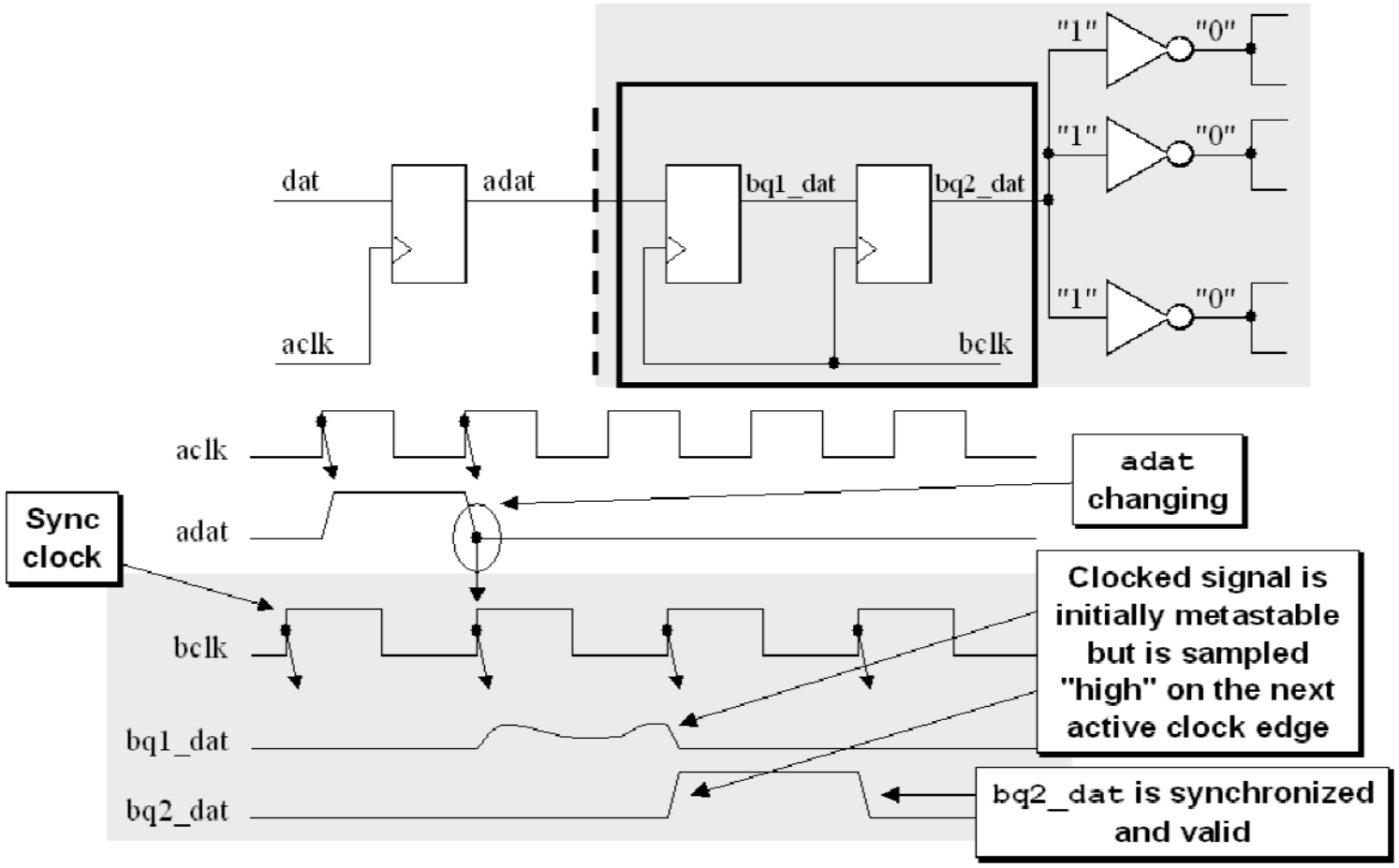
Synchronization Techniques

- **Open-loop and Close-loop Synchronization**

- **For Single-bit control signal synchronization :**
  - 2-FF or 3-FF Synchronizer (open-loop)

- **For Multi-bit data bus synchronization :**
  - Handshake Mechanism (closed loop)
  - Asynchronous FIFO (open loop)

2-FlipFlop Synchronizer to Address Metastability



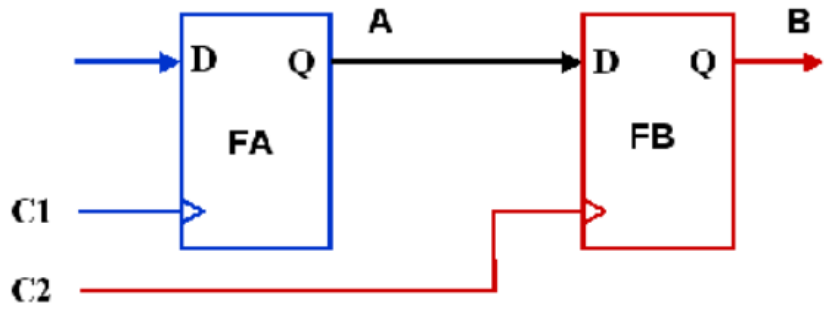
Note : Multi-Flop synchronizers allow sufficient time for the oscillations to settle down and ensure that a stable output is obtained in the destination domain

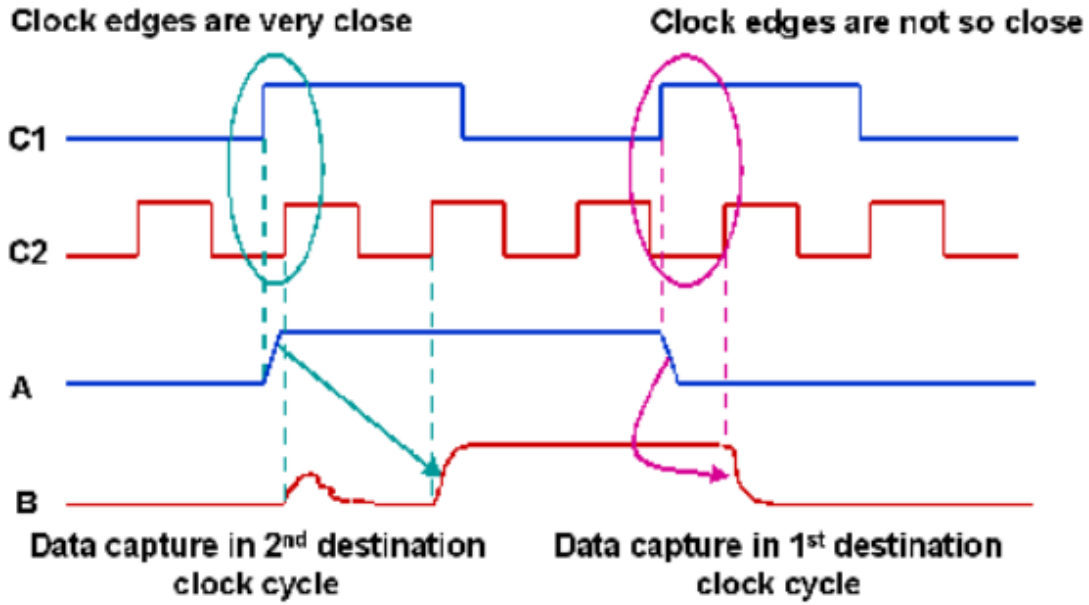
- First flipflop (**FF1**) samples the asynchronous input signal into the new clock domain (**bclk**)
- Waits for full clock cycle to permit any metastability on the synchronizer stage-1 output signal (**bq1_dat**) to decay, then the stage-1 signal is sampled by the same clock(**bclk**) into a second stage flipflop (**FF2**)
- The stage-2 signal (**bq2_dat**) is now a stable and valid signal synchronized and ready for distribution within the new clock domain

Source of this slide courtesy of Clock Domain Crossing publication from Clifford E. Cummings Sunburst Design

Signal Crossing Slow Clock Domain to Fast Clock Domain

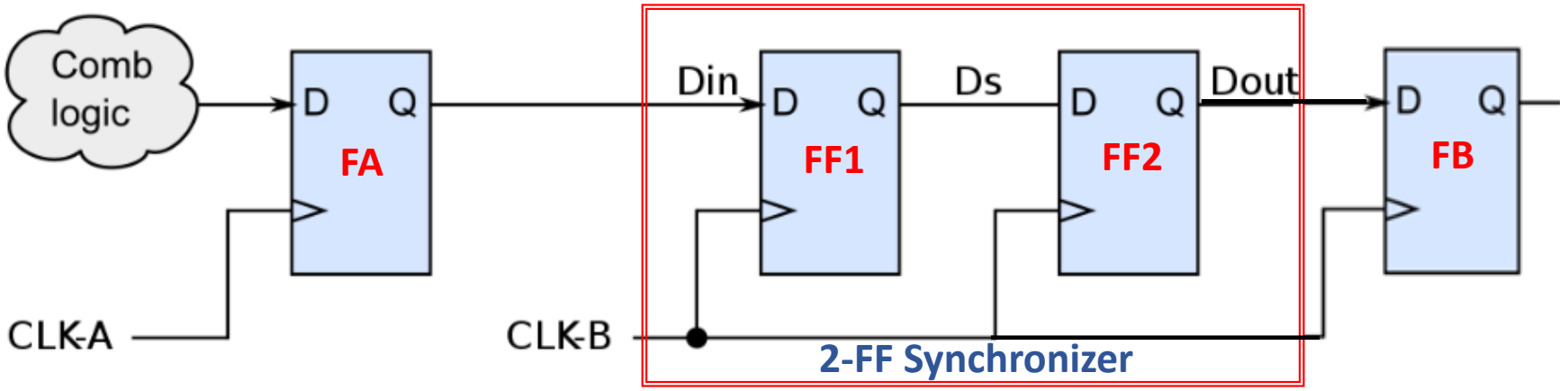
- **Scenario -1 : Flipflop-A clock frequency < Flipflop-B clock frequency**

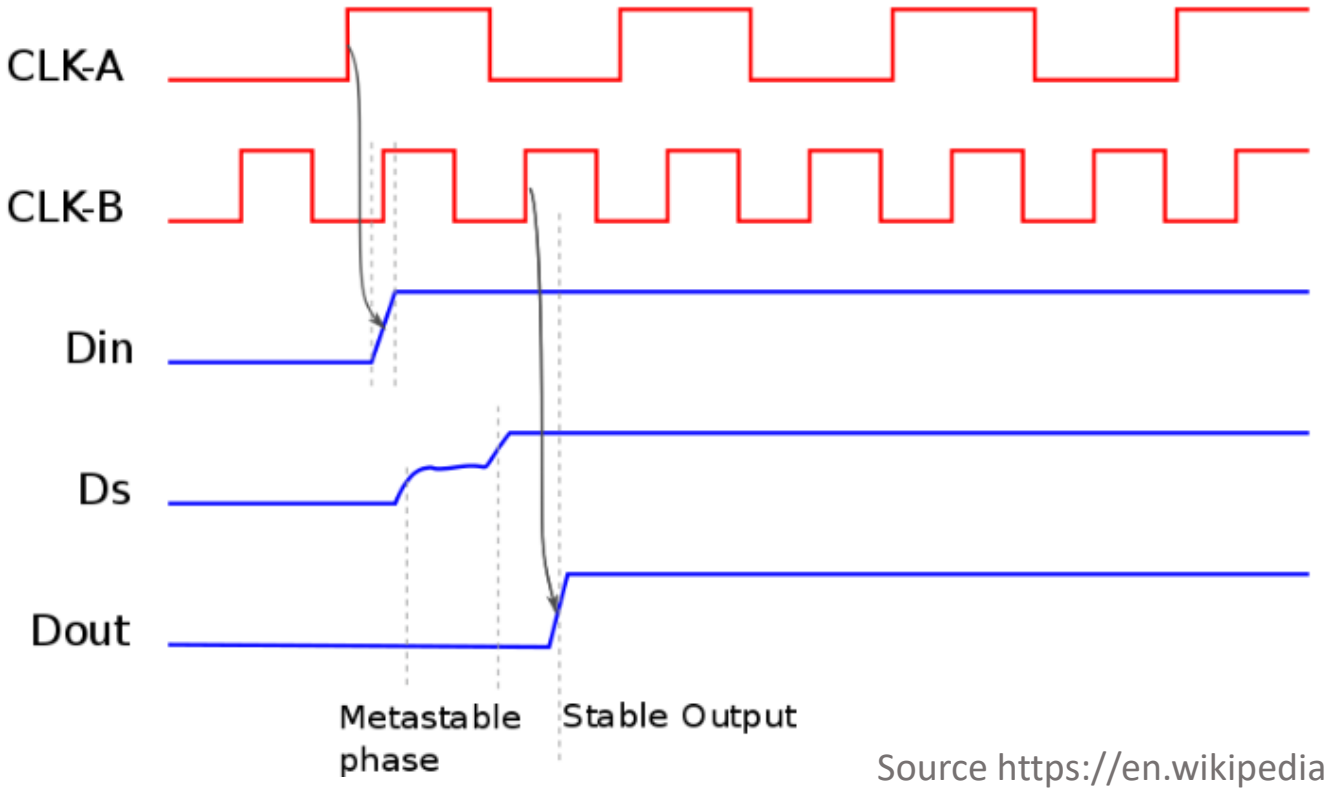




- If the transition on signal A happens very close to the active edge of clock C2, it could lead to setup or hold violation at the destination flop "FB“ and “FB” will enter metastable state.
- Output signal B from FA will be unstable and may or may not settle down to some stable value before the next clock edge of C2 arrives
- Unstable signal B fanout cones may read different values, and may cause the design to enter into an unknown functional state, leading to functional issues in the design
- **To address such slow to fast clock domain crossing scenario, two or three flipflop synchronizer can be used !**

2-FlipFlop Synchronizer For Slow to Fast Clock Domain



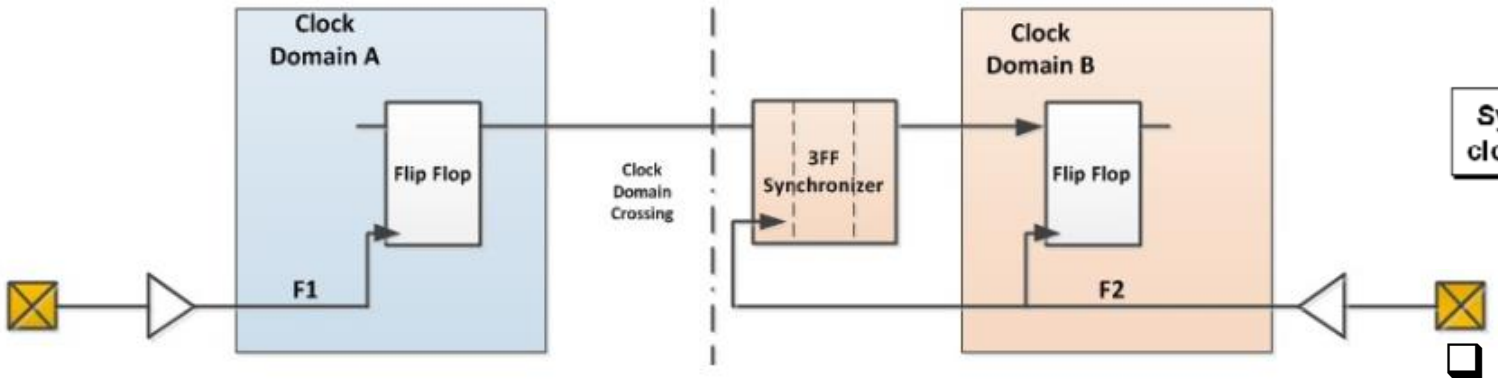


- CLK-A frequency is slower than CLK-B
- Din entering into CLK-B domain goes through 2-FF synchronizer stage (FF1, FF2)
- 2-FF synchronizer will operate on receiving clock which is CLK-B
- Even if Din changes close to CLK-B clock edge violating setup violation of FF1, causing metastable output Ds, output of second stage of synchronizer (FF2) will be stable.
- If Din changes fast and frequently, sometimes 2-FF synchronizer is not enough for metastable output to settle. In such cases 3rd FF can be added (i.e.3-FF synchronizer)

Source https://en.wikipedia.org/wiki/Metastability_(electronics)

2-FlipFlop and 3-FlipFlop Synchronizer

3-FF Synchronizer



$$MTBF = \frac{1}{f_{clk} * f_{data} * X}$$

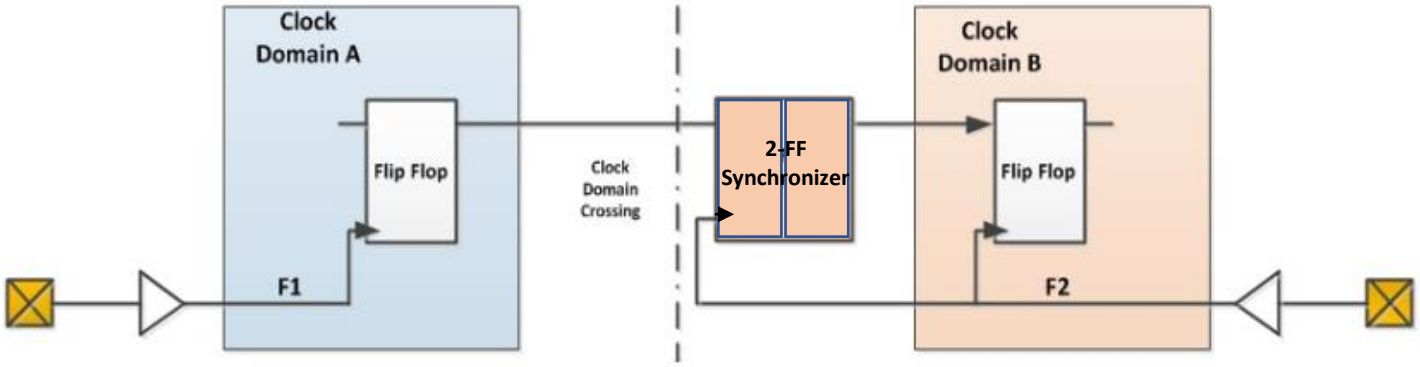
Other factors

Synchronizing clock frequency

Data changing frequency

Figure 4 - Primary contributing factors to short MTBF values

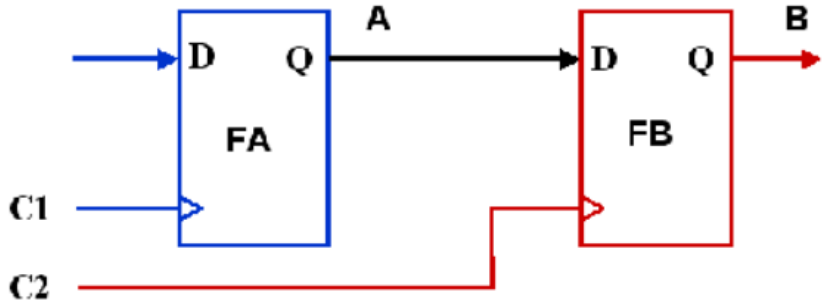
2-FF Synchronizer

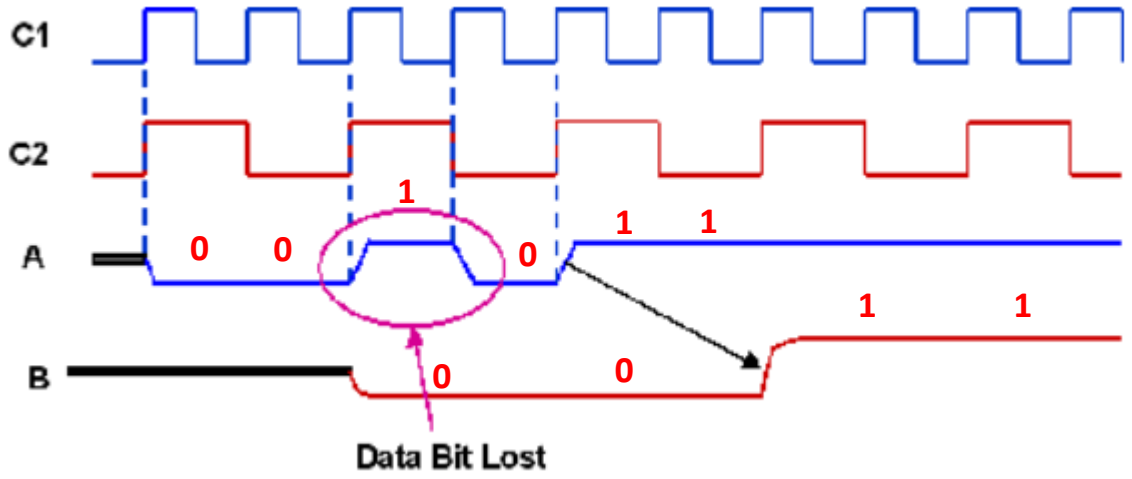


- **Synchronizers must be designed to reduce the chances system failure due to metastability**
- **Synchronizer requirements**
  - Reliable [high MTBF]
  - Low latency [works as quickly as possible]
  - Low power/area impact
- **For some designs 3-FF synchronizer might be required if source frequency is high and input data is changing fast**
- **Can increase MTBF by adding more series stages**
  - 3-FF synchronizer can have higher MTBF

Signal Crossing Fast Clock Domain to Slow Clock Domain

- **Scenario-2 : Flipflop-A clock frequency > Flipflop-B clock frequency**

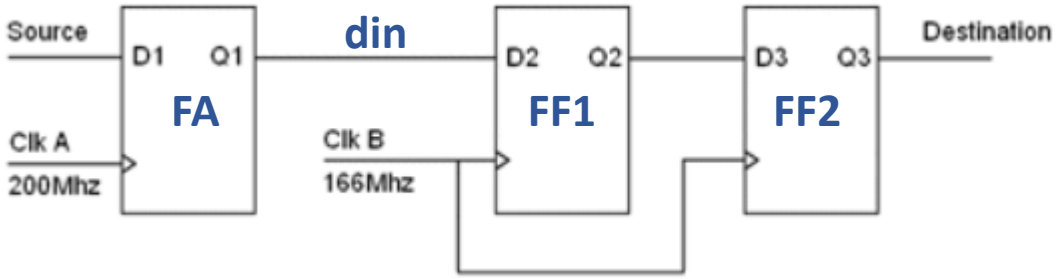


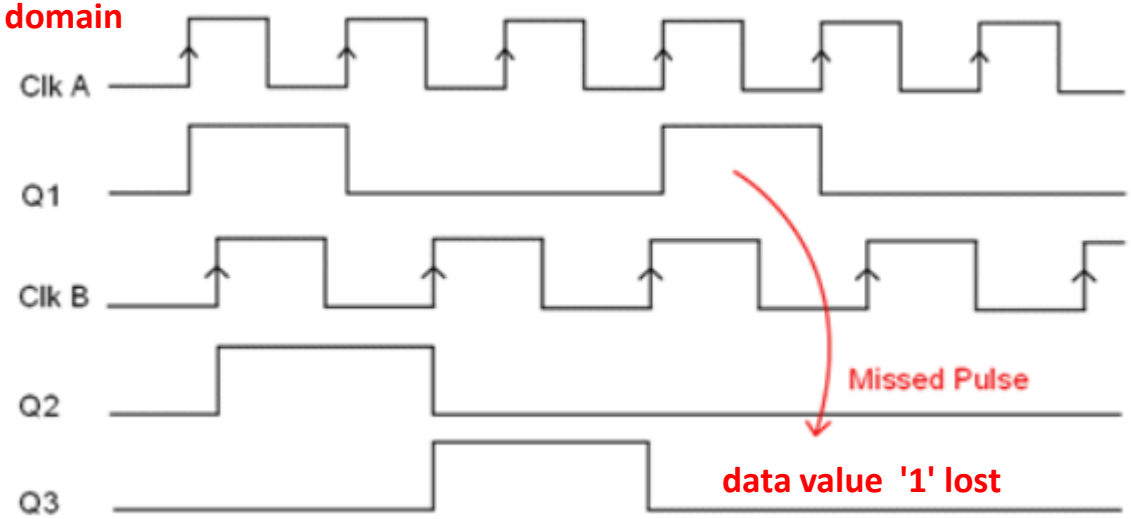


- C1 is two times faster than C2 and there is no phase difference between C1 and C2
- If signal A is changing rapidly, say for example signal A sequence is "**001011**", output from FB in C2 clock domain will be "**0011**".
  - Here the third data value in the input sequence which is **"1"** is lost → **data losss**
  - For some circuits data loss is not acceptable
- **Two or three Flipflop synchronizer technique will avoid data loss, instead either of the technqiues can be used to avoid data loss when data is sent from fast to slow clock domain :**
  - Storage between FA and FB (Asynchronous FIFO)
  - Handshake Synchronization technique between FA and FB

2-FF Synchronizer for Signal crossing Fast Clock to Slow Clock Domain

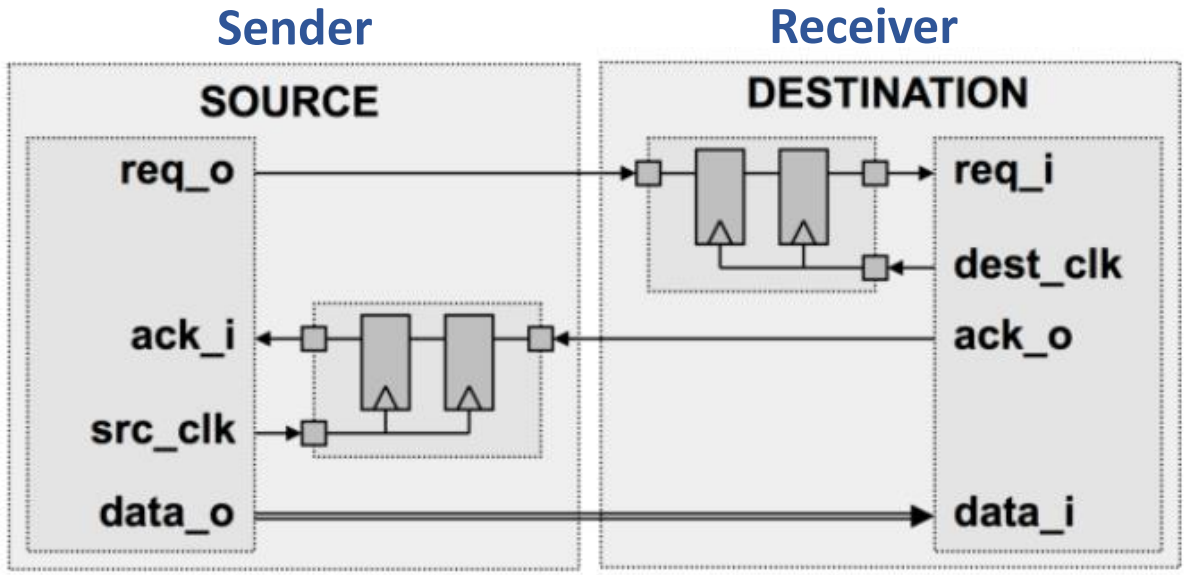
Example of data Loss when data flowing from fast to slow to fast clock domain

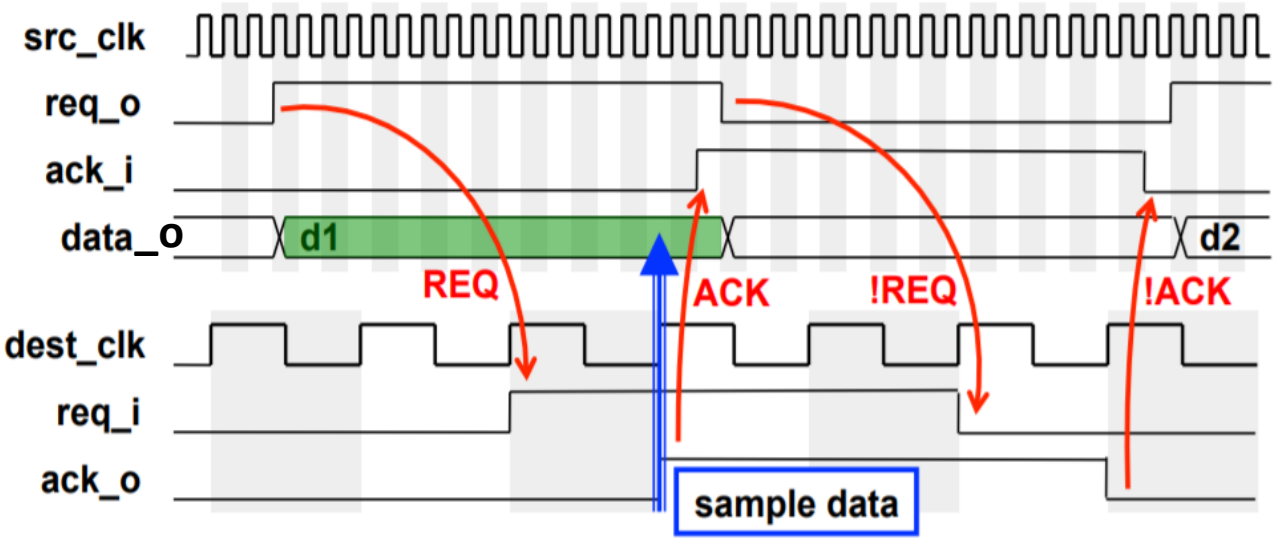




- Consider, signal crossing from source clock domain ClkA to Destination ClkB domain where ClkA is faster than ClkB
- ClkA is 200 Mhz and Clk B is 166 MHz
- Signal is Crossing Fast to Slow clock domain through 2-FF synchronizers (FF1 and FF2)
- Adding 2-FF or 3-FF synchronizer will still result in data loss when signal crossed fast to slow clock domain as seen in the waveform above
- If source signal “din” width entering FF1 can be guaranteed to be **1.5** times of receiving clock ClkB then 2-FF or 3-FF synchronizer still can be used for signals crossing fast to slow clock domain
- If source signal cannot be guaranteed to 1.5 times of receiving clock and if the data loss is not acceptable for subsequent circuit in destination clock domain **then 2-FF or 3-FF open ended synchronization technique should not be deployed !**

Handshake Synchronizer For Multi-bit Signal Crossing Fast to Slow Clock Domain





- For many open-ended data-passing applications, a simple **req-ack** handshaking sequence is sufficient
- The sender places data onto a data bus (**data_o**) and then synchronizes a "**req_o**" signal (request) to the receiving clock domain.
- When the "**req_o**" signal is recognized in the destination clock domain, the receiver clocks the data_o into a register (the data should have been stable for at least two/three sampling clock edges in the destination clock domain)
- Receiver then passes an "**ack_o**" signal (acknowledgement) through a synchronizer to the sender.
- When the sender recognizes the synchronized "**ack_o**" signal, the sender can change the value being driven onto the data bus (**data_o**)

Homework-8 : Handshake Synchronizer

❑ Requirements for handshake synchronizer

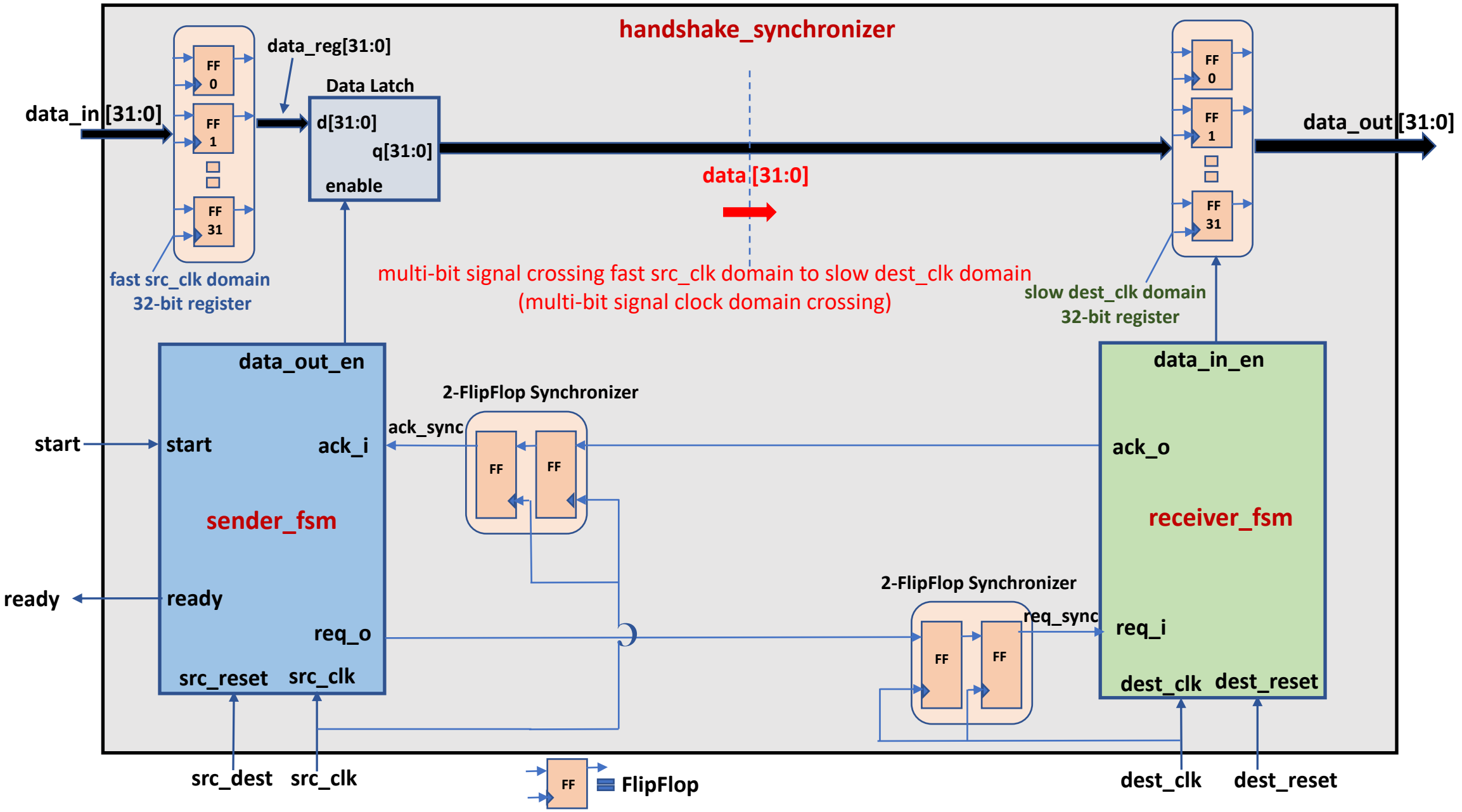
- Design handshake synchronizer to send data form fast clock domain to slower clock domain without dropping any data
- Create **sender_fsm** and **receiver_fsm** modules
- Review flipflop_synchronizer code provided in lab folder
- Create **handshake_synchronizer** module and instantiate **sender_fsm, receiver_fsm, flipfliop synchronizer** and make connections
  - Refer to handshake_synchronizer block diagram on next slide when making connections between the modules
  - Add latch after the source data register to hold data until data is captured by the destination register
  - Size of data to be sent and captured = 32 bits
- sender_fsm and receiver_fsm should implement 4-phase req-ack handshake protocol (see slide on this requirement)
- Review resource utilization including total flipflops and combination ALUT's in report
- Simulate **handshake_synchronizer** with testbench provided
  - Testbench generates src_clk with time period of 20ns and dest_clk with time period of 124 ns (i.e. src_clk is around 6 times slower than dest_clk).
  - Note : Higher the time period lower the frequency (f = 1 / T). Since src_clk time period is lower its frequency is higher
- Review simulation waveform and explain results of **handshake_synchronizer**
  - There is no self-checker implemented in testbench so review input data_in and data_out values of handshake_synchronizer in simulation waveform
  - Ensure each data value sent by sender_fsm are successfully captured by receiver_fsm
- Primary port list for **sender_fsm, receiver_fsm, handshake_synchronizer** modules :
  - **handshake_synchronizer** : **input** src_clk, src_reset, dest_clk, dest_reset, start, **input** [31:0] data_in, **output** ready **output**[31:0] data_out
  - **sender_fsm** : **input** src_clk, src_reset, start, ack_i, **output** data_out_en, ready, req_o
  - **receiver_fsm** : **input** dest_clk, dest_reset, req_i, **output** data_in_en, ack_o

Homework-8 : Report Submission and Lab8 Folder

- **Homework Report should include :**
  - SystemVerilog design code snapshot for :
    - handshake_synchronizer, sender_fsm, receiver_fsm, flipflop_synchronizer modules
  - Synthesis resource usage snapshot generated from Quartus for handshake_synchronizer top level module
  - Simulation snapshot and explain simulation result of handshake_synchronizer
  - Explanation of FPGA resource usage in report is not required.

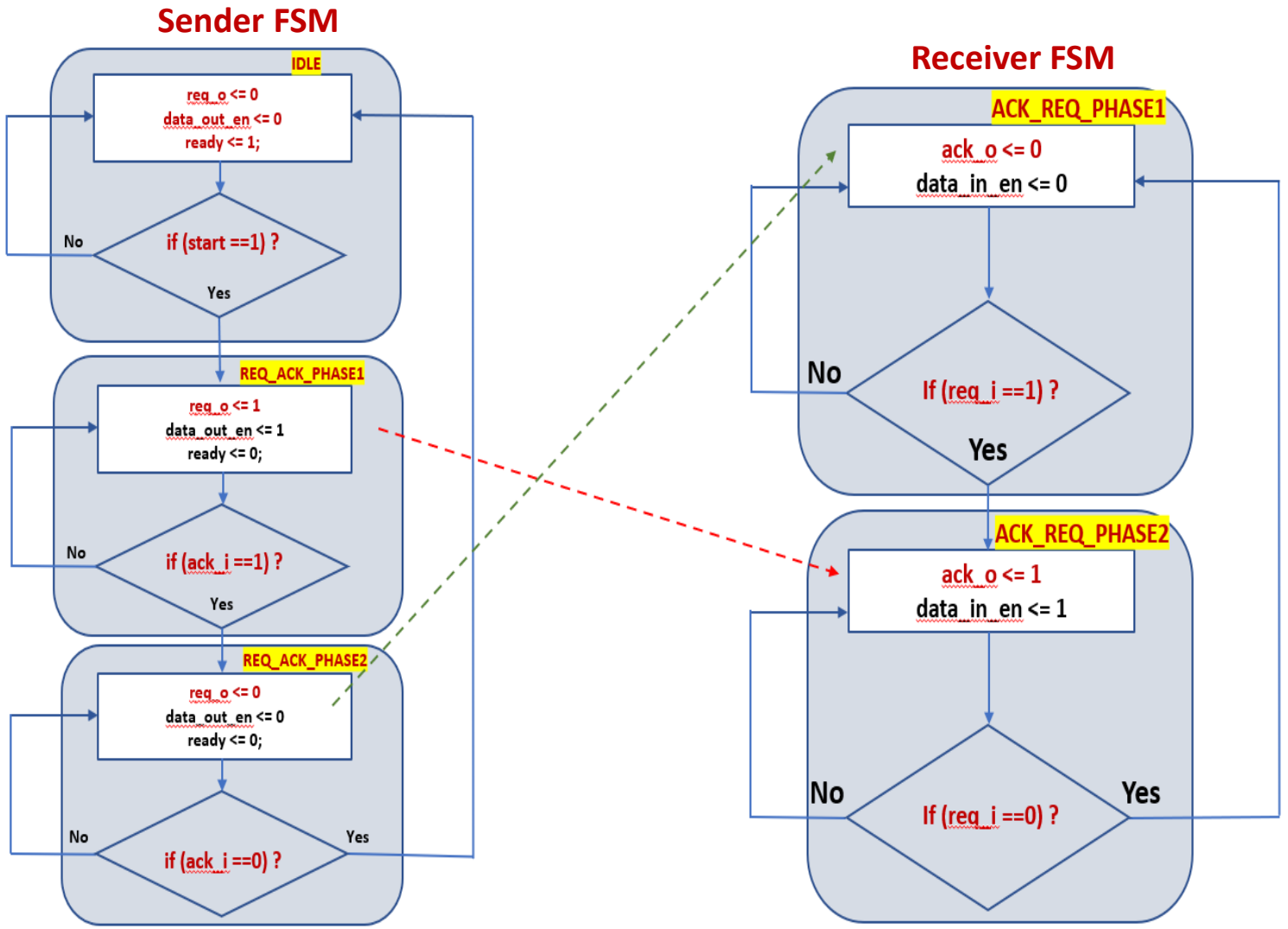
- **Lab8 folder includes :**
  - Handshake_synchronizer folder which includes
    - Design template code for handshake_synchronizer, sender_fsm, receiver_fsm, flipflop_synchronizer modules
    - handshake_synchronizer full testbench code
  - For learning purpose, student can change the stimulus in initial block in testbench files

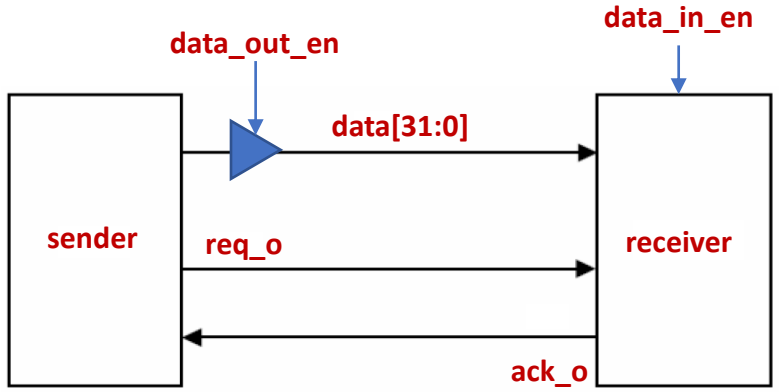
Homework-8 : Handshake Synchronizer



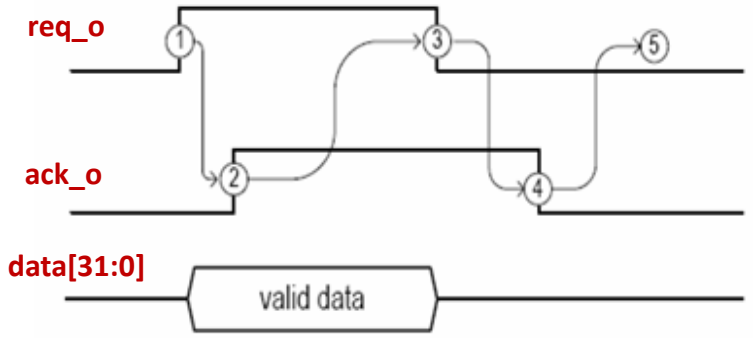
Homework-8 : Handshake Synchronizer

- 4-phase req-ack handshake protocol between sender_fsm and receiver_fsm

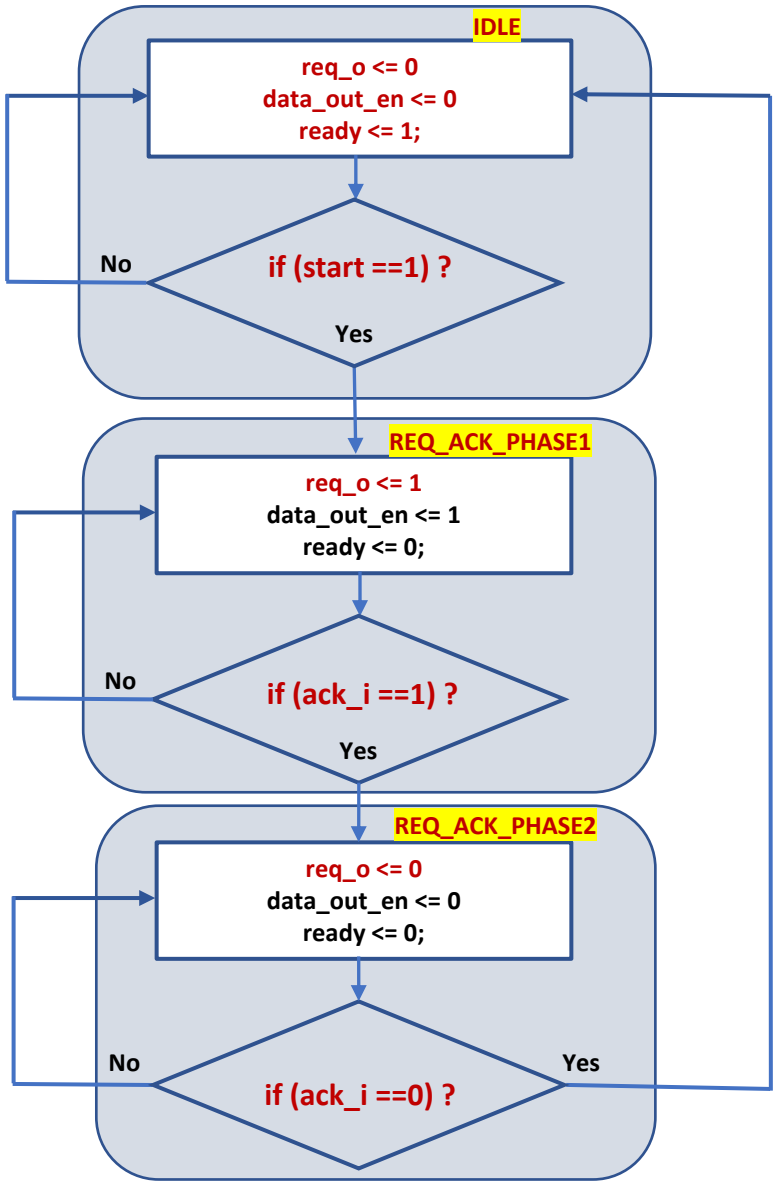




4-phase req-ack handshake protocol

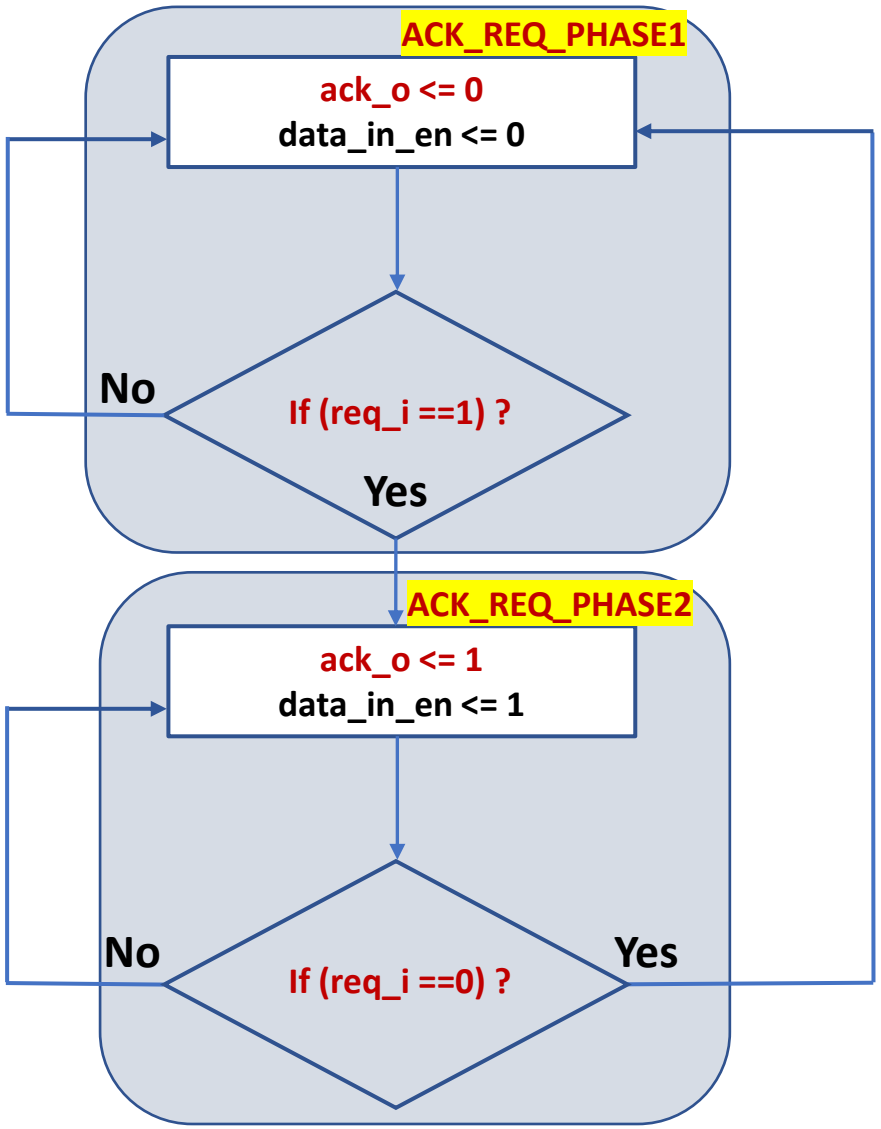


Homework-8 : Handshake Sender FSM



- **Handshake Sender FSM**
  - **Purpose :** Sends data from faster src_clk domain to slower dest_clk domain using req ack handshake protocol. Holds data until dest_clk domain receiver_fsm has not acknowledged acceptance of data. Once data is accepted by receiver_fsm, then sender_fsm gets ready to send next data to dest_clk domain
  - **IDLE State :**
    - Asserts ready = 1 indicating testbench that handshake sender_fsm is ready to receive next 32-bit data
    - Waits for start == 1 from testbench, which indicates next 32-bit data is available to be sent to dest_clk domain. And then move to REQ_ACK_PHASE1
  - **REQ_ACK_PHASE1 :**
    - Asserts req_o amd data_out_en to '1' which indicates request is available for receiver_fsm to accept
    - Then waits for ack_i == 1 from receiver_fsm and then moves to REQ_ACK_PHASE2 once request acceptance has been acknowledged
  - **REQ_ACK_PHASE2 :**
    - De-asserts req_o amd data_out_en to '0' since receiver_fsm has captured 32-bit data
    - Then waits for ack_i == 0 from receiver_fsm which is indication that receiver_fsm is now ready to receive next 32 bit data from src_clk domain and then moves to IDLE state

Homework-8 : Handshake Receiver FSM



❑ Handshake Receiver FSM

- **Purpose :** Receive data from fast src_clk domain and capture it in slow dest_clk domain register (flipflop) using req ack handshake protocol. Provides acknowledgement to receiver_fsm once data is captured in the destination register (flipflop)

- **ACK_REQ_PHASE1 :**
  - De-asserts ack_o amd data_in_en to '0' and then waits for req_i == 1
  - This indicates to sender_fsm that it is ready to receive next 32 bit data and then moves to ACK_REQ_PHASE2

- **ACK_REQ_PHASE2 :**
  - Asserts ack_o amd data_in_en to '1' which indicates 32-bit data from src_clk domain sender_fsm has been captured in dest_clk domain
  - Then waits for req_i == 0 from handshake sender_fsm and then moves to REQ_ACK_PHASE1 to complete acknowledgement handshake protocol

Homework-8 : Handshake Synchronizer

❑ Reference simulation waveform for handshake_synchronizer module

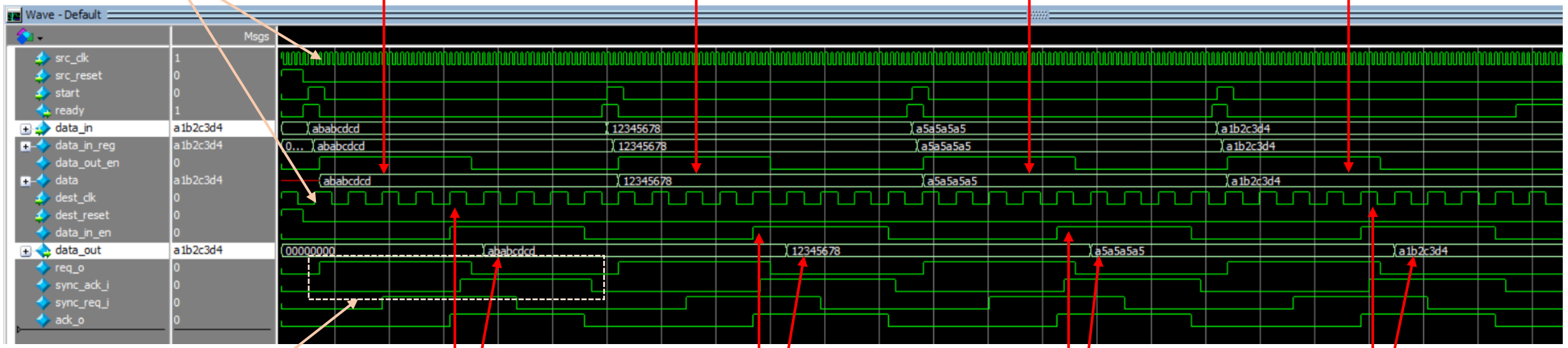
src_clk = 20 ns clock period
dest_clk = 124 ns clock period
dest_clk is ~6 times slower than src_clk

First 32-bit data 32'hababcdcd sent from fast src_clk domain by sender_fsm to receiver with data_out_en = 1

Second 32-bit data 32'h12345678 sent from fast src_clk domain by sender_fsm to receiver with data_out_en = 1

Third 32-bit data 32'ha5a5a5a5 sent from fast src_clk domain by sender_fsm to receiver with data_out_en = 1

Fourth 32-bit data 32'ha1b2c3d4 sent from fast src_clk domain by sender_fsm to receiver with data_out_en = 1



4-phase req-ack handshake between sender_fsm and receiver_fsm

First 32-bit data 32'ha5ababcdcd captured by in slow dest_clk domain by receiver_fsm when data_ie_en = 1 and available on data_out 1 cycle later

Second 32-bit data 32'h12345678 captured by in slow dest_clk domain by receiver_fsm when data_ie_en = 1 and available on data_out 1 cycle later

Third 32-bit data 32'ha5a5a5a5 captured by in slow dest_clk domain by receiver_fsm when data_ie_en = 1 and available on data_out 1 cycle later

Fourth 32-bit data 32'ha5a1b2c3d4 captured by in slow dest_clk domain by receiver_fsm when data_ie_en = 1 and available on data_out 1 cycle later

References

- **Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog Clifford E. Cummings Sunburst Design, Inc**
  - http://www.sunburst-design.com/papers/CummingsSNUG2008Boston_CDC.pdf

- **About Clock Domain Crossing**
  - https://en.wikipedia.org/wiki/Clock_domain_crossing